

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-a2f7f506caeb082d8.jsonl`
- SHA-256: `86ba61ceb80751027c22a333b7208ce017385cdb660be8911ffa6e7b6a79cddb`
- Source modified: `2026-06-22T15:07:19+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_f39953fd-98d`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T15:05:03.846Z)

Adversarially verify this finding by reading the cited code in C:/Users/avidu/Projects/bhi-cloudrun (run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`). Real defect to fix, or false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: dispatch-correctness

Severity: major

Title: Partial cloud ingest leaks rows + breaks destroy-after-confirm (no try/except around _ingest_remote_segments)

File: backend/main.py:518-523

Detail: After a successful cloud job, `\_ingest\_remote\_segments(video\_id, result.outputs\_uri, storage\_dict)` is called with NO surrounding try/except. It writes new play_segments rows one-by-one via db.add_play_segment, and any failure AFTER it has written some rows but before `\_finalize\_reingest` runs (e.g. result.outputs_uri/intervals.csv missing so storage.fetch raises mid-stream-impossible, OR a transient add_play_segment, OR the intervals.csv object simply not present so the initial fetch raises) leaves the partial NEW rows (id > play_wm) in the DB. The exception propagates uncaught through `\_maybe\_dispatch\_remote` -> `\_execute\_processing` (line 606 is OUTSIDE the inner try that only wraps segmenter.process_video) ->

job_worker._run_claimed_job, which merely calls mark_job_failed and does NOT restore segments.

Net: on a re-ingest the video now has BOTH the prior good rows AND orphaned partial cloud rows ? a violation of the destroy-after-confirm contract the in-process path carefully upholds (main.py:614-619 prunes on raise). The missing-intervals.csv case is realistic: the worker writes report.json + intervals.csv, but if only report.json landed (push interrupted) the fetch of intervals.csv raises.

Fix: Wrap the ingest in try/except like the in-process segmenter: on any exception call db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm) (drop the partial new rows, keep prior good rows) then return _failed(...). i.e. reuse the _failed() helper around _ingest_remote_segments + the segments query.

2. assistant (2026-06-22T15:05:06.906Z)

I'll verify this finding by reading the cited code in the bhi-cloudrun repository.

3. user (2026-06-22T15:05:09.836Z)

---DIFF---

```
backend/main.py | 100 ++++++
tests/test_api_cloud_dispatch.py | 212 ++++++
2 files changed, 312 insertions(+)
```

4. assistant (2026-06-22T15:05:12.216Z)

Let me read the relevant code in main.py.

5. user (2026-06-22T15:05:14.021Z)

```
diff --git a/backend/main.py b/backend/main.py
index 4e5a19a..04ffb67 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -1,10 +1,12 @@
import os
import sys
import argparse
+import csv
import json
import logging
import shutil
import socket
+import tempfile
import threading
import uuid
import webbrowser
@@ -436,6 +438,94 @@ def download_reel(filename: str):
    return FileResponse(path, media_type=media, filename=name)

+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
+    bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
+    produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
+    (cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                   "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                             play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
+    ``deployment.compute_target
+    == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
+    intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
```

```

``None`` to
+   fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns
``None``
+   for every non-``cloud_run`` target, so the in-process path stays byte-identical.""
+   from backend.compute import ComputeJobSpec, get_compute_backend
+
+   compute = get_compute_backend(global_config)
+   if compute is None:
+       return None # default-OFF ? run the in-process segmenter below (unchanged)
+
+   storage_cfg = getattr(global_config, "storage", None)
+
+   def _failed(msg: str) -> Dict[str, Any]:
+       # Shape-match the in-process segmenter-fail branch + restore prior good rows (none
written yet).
+       db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+       return {"video_id": video_id, "status": "failed", "message": msg,
+               "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+   # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+   try:
+       input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+   except ValueError as e:
+       return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+   if input_ref.backend == "local":
+       return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+
+                           "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+   out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+   if not out_root:
+       return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+
+                           "? that's where the Cloud Run job writes the result.")
+
+   notify("dispatching (cloud_run)")
+   logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+   spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+   try:
+       result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
+   except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+       logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+       return _failed(f"cloud dispatch error: {e}")
+   if result.returncode != 0:
+       return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+   notify("ingesting (cloud_run)")
+   storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+   _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+   # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+   segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+   _finalize_reingest(video_id, segments, play_wm, cand_wm)
+   return {"video_id": video_id, "status": "success",
+           "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
+           "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+   def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:

```

""The full hash ? validate ? segment ? report pipeline for one video.

```
@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
    # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
    # if the run yields segments; otherwise drop the new partial rows and keep the old.
    play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+    # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
+    # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+    # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+    # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
+    # they're dropped on the cloud path in this slice ? a documented follow-up.)
+    remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+    if remote is not None:
+        segmentation_ran = True
+        response = remote
+        return response
    segmenter = segmenter_cls(db_instance, global_config)
    try:
        result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
```

6. assistant (2026-06-22T15:05:16.320Z)

Now let me read the surrounding context in main.py to understand the full control flow, especially the in-process path's try/except and the watermark/prune semantics.

7. user (2026-06-22T15:05:17.420Z)

```
560     seg_name = req.segmenter or default_seg
561     segmenter_actual = None
562     segmentation_ran = False
563     response: Optional[Dict[str, Any]] = None
564     try:
565         if failures:
566             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
567             # status; it no longer destroys the video's prior good segments.
568             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
569             response = {
570                 "video_id": video_id,
571                 "status": "failed",
572                 "message": "Video failed validation checks.",
573                 "results": [r.model_dump() for r in val_results],
574             }
575             return response
576
577         # 2. Run segmenter & indexer
578         logger.info("[API] Video passed checks. Segmenting and indexing...")
579         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
580
581         segmenter_cls = segmenter_registry.get(seg_name)
582         if not segmenter_cls:
583             # Submit-time validation already 400s unknown names; this is the defensive
584             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
585             available = list(segmenter_registry.list_available().keys())
586             response = {
587                 "video_id": video_id,
```

```

588         "status": "failed",
589         "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
590         "results": [r.model_dump() for r in val_results],
591         "play_segments": [],
592     }
593     return response
594
595     logger.info(f"[API] Using segmenter: {seg_name}")
596     notify(f"segmenting ({seg_name})")
597     segmenter_actual = seg_name
598     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
599     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
600     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
601     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
602     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
603     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
604     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
605     # they're dropped on the cloud path in this slice ? a documented follow-up.)
606     remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
607     if remote is not None:
608         segmentation_ran = True
609         response = remote
610         return response
611     segmenter = segmenter_cls(db_instance, global_config)
612     try:
613         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
614     except Exception:
615         # A raising segmenter must not leave half-written rows: restore the pre-run
616         # state (same destroy-after-confirm contract as the Failure branch), then
let
617         # the job worker record the failure.
618         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
619         raise
620     segmentation_ran = True
621
622     if isinstance(result, Failure):
623         logger.error(f"[API] Segmenter failed: {result.message}")
624         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
625         response = {
626             "video_id": video_id,
627             "status": "failed",
628             "message": f"Segmenter '{seg_name}' failed: {result.message}",
629             "results": [r.model_dump() for r in val_results],
630             "play_segments": [],
631         }
632         return response
633
634     segments = result.value if hasattr(result, "value") else result
635     notify("finalizing")
636     _finalize_reingest(video_id, segments, play_wm, cand_wm)
637
638     response = {
639         "video_id": video_id,
640         "status": "success",
641         "message": f"Successfully indexed {len(segments)} play segments using
'{{seg_name}}' segmenter.",

```

```

642         "results": [r.model_dump() for r in val_results],
643         "play_segments": segments,
644     }
645     return response
646 finally:
647     # Always emit the per-video observability report (next to the video, or to
648     # report.output_dir). Never raises. Surface the paths in the response.
649     paths = emit_indexer_report(
650         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
651         val_results=val_results, val_context=val_context,
652         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
653         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
654     )
655     if paths and isinstance(response, dict):
656         response["reports"] = paths
657
658
659     @app.post("/api/process", status_code=202)

```

8. assistant (2026-06-22T15:05:21.246Z)

Now I have the full picture. Let me verify the key claim about the in-process path's exception handling. The in-process path at line 612-619 wraps `segmenter.process\_video` in a try/except that prunes on raise. Now let me examine the `\_maybe\_dispatch\_remote` flow precisely, focusing on where `\_ingest\_remote\_segments` is called (lines around 510-523 in the new code, which I need to map to the actual line numbers).

Let me look at the exact lines the finding cites (518-523).

9. user (2026-06-22T15:05:22.521Z)

```

486     storage_cfg = getattr(global_config, "storage", None)
487
488     def _failed(msg: str) -> Dict[str, Any]:
489         # Shape-match the in-process segmenter-fail branch + restore prior good rows
(none written yet).
490         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
491         return {"video_id": video_id, "status": "failed", "message": msg,
492             "results": [r.model_dump() for r in val_results], "play_segments": []}
493
494     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
495     try:
496         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
497     except ValueError as e:
498         return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
499     if input_ref.backend == "local":
500         return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
this machine "
501             "can't be read by the Cloud Run job. Put the video in your bucket
first.")
502     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
503     if not out_root:
504         return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
505             "? that's where the Cloud Run job writes the result.")
506
507     notify("dispatching (cloud_run)")
508     logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
509     spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name)
510     try:
511         result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to

```

```

completion
512         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
513             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
514             return _failed(f"cloud dispatch error: {e}")
515         if result.returncode != 0:
516             return _failed(f"cloud job failed (returncode {result.returncode}).")
517
518         notify("ingesting (cloud_run)")
519         storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
{}
520         _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
521         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
522         segments = [s for s in db_instance.query_play_segments(video_id, {}) if
int(s.get("id", 0)) > play_wm]
523         _finalize_reingest(video_id, segments, play_wm, cand_wm)
524         return {"video_id": video_id, "status": "success",
525               "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
526               "results": [r.model_dump() for r in val_results], "play_segments": segments}
527
528
529     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
530         """The full hash ? validate ? segment ? report pipeline for one video.

```

10. assistant (2026-06-22T15:05:26.979Z)

The cited lines 518-523 match exactly. The finding is precise: `\_ingest\_remote\_segments` at line 520 is called with no try/except around it. Now let me verify the critical claims about what happens on exception. Let me check:

1. `\_finalize\_reingest` semantics (does it prune old rows?)
2. `\_prune\_segments` semantics
3. The `add\_play\_segment` writes rows one-by-one (already confirmed in the diff)
4. The propagation path through `\_execute\_processing` and the job worker

Let me look at `\_finalize\_reingest`, `\_prune\_segments`, and the job worker.

11. user (2026-06-22T15:05:28.234Z)

```

285: def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
521:     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops
the old ones.
523:     _finalize_reingest(video_id, segments, play_wm, cand_wm)
636:         _finalize_reingest(video_id, segments, play_wm, cand_wm)
1162:             _finalize_reingest(video_id, segments, play_wm, cand_wm)

```

12. user (2026-06-22T15:05:31.076Z)

```

285     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
286         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
287
288         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
289         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
290         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
291         """
292         if segments:
293             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
294         else:
295             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
296

```

```

297
298     # --- API Endpoints ---
299     @app.get("/api/config")
300     def get_config():
301         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
302         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
303         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
304         # by tests/test_redact.py + the /api/config redaction test).
305         if global_config is None:
306             return {}
307         return redact_secrets(global_config.model_dump(mode="json"))
308
309     @app.get("/api/health")
310     def health():
311         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
312         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
313         confirm the engine is reachable and which detector is wired. Never raises."""
314         sport = getattr(global_config, "sport", None)

```

13. assistant (2026-06-22T15:05:34.304Z)

This confirms the destroy-after-confirm contract. Now let me check `prune\_segments` to confirm `play\_after` semantics (drops rows with id > watermark).

14. user (2026-06-22T15:05:35.252Z)

```

backend/infrastructure/database.py:438:     def prune_segments(self, video_id: str, *, play_upto:
Optional[int] = None,

```

15. user (2026-06-22T15:05:37.815Z)

```

438         def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
439                             play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
440                             cand_after: Optional[int] = None) -> None:
441             """Delete play/candidate segments by id range (events cascade from
play_segments).
442
443             Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
444             successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
445             and keep the OLD ones when the run failed/produced nothing (`*_after`).
446             """
447             with self._connect() as conn:
448                 cur = conn.cursor()
449                 if play_upto is not None:
450                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
451                 if play_after is not None:
452                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
453                 if cand_upto is not None:
454                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
455                 if cand_after is not None:
456                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
457                 conn.commit()
458
459         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]

```



```

= None) -> Optional[int]:
460         try:
461             with self._connect() as conn:
462                 cursor = conn.cursor()
463                 cursor.execute(
464                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
465                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
466                 )
467                 conn.commit()
468                 return cursor.lastrowid
469         except Exception as e:
470             logger.error(f"Failed to add play segment: {e}")
471             return None
472
473     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
474         return self._execute_write(
475             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
476             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
477             "Failed to add event",
478         )
479
480     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
481                               chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
482                               stats: Optional[Dict[str, Any]] = None,
483                               detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
484         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
485         detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
486         try:
487             with self._connect() as conn:
488                 cursor = conn.cursor()
489                 cursor.execute(
490                     """INSERT INTO candidate_segments
491                     (video_id, detector, chunk_index, start_time, end_time, duration,
492                     confidence, stats, detection_params)

```

16. assistant (2026-06-22T15:05:45.316Z)

Important detail here: `add\_play\_segment` itself does NOT raise on failure ? it catches `Exception`, logs, and returns `None` (line 469-471). So the "transient add_play_segment" failure mode in the finding is NOT a way the exception propagates; that's a false sub-claim. Each `add\_play\_segment` commits its own row (line 467), so rows are written one-by-one (this part is true).

The realistic exception path is `storage.fetch` at line 520? `\_ingest\_remote\_segments` line. But note: `storage.fetch` happens BEFORE any `add\_play\_segment` is called (the fetch is at the top of `\_ingest\_remote\_segments`, before the loop). Let me re-examine. If `intervals.csv` is missing, `storage.fetch` raises BEFORE any row is written ? so no partial rows. The partial-row leak requires a failure mid-loop (after some `add\_play\_segment` calls).

Since `add\_play\_segment` swallows its own exceptions, a mid-loop raise can only come from `csv.DictReader` iteration or the `float()`/`int()` conversions ? but those are inside the per-row `try/except` (line guarded). Let me re-read `\_ingest\_remote\_segments` carefully.

17. user (2026-06-22T15:05:46.470Z)

```

441     def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) ->
int:

```

```

442         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
443         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
444         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
445         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
446         backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
447         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
448         ref = storage.parse_uri(uri)
449         dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or
"intervals.csv")
450         local = storage.fetch(uri, dst, config=storage_config)
451         n = 0
452         with open(local, newline="", encoding="utf-8") as f:
453             for row in csv.DictReader(f):
454                 try:
455                     start, end = float(row["start"]), float(row["end"])
456                 except (KeyError, TypeError, ValueError):
457                     continue
458                 meta: Dict[str, Any] = {"source": "cloud_run",
459                                         "ending_reason": (row.get("ending_reason") or
"").strip()}
460                 sc = (row.get("shot_count") or "").strip()
461                 if sc:
462                     try:
463                         meta["shot_count"] = int(float(sc))
464                     except ValueError:
465                         meta["shot_count"] = sc
466                 db_instance.add_play_segment(video_id, start, end, metadata=meta)
467                 n += 1
468         return n

```

18. assistant (2026-06-22T15:05:56.036Z)

Now I can analyze each exception scenario the finding claims, precisely:

****Scenario A: `intervals.csv` missing ? `storage.fetch` raises (line 450).**** This happens BEFORE the loop, so NO `add\_play\_segment` rows are written yet. The exception propagates uncaught, but there are zero partial rows. So this does NOT leak partial rows. The finding explicitly calls this case "realistic" and claims it leaves partial rows ? but at this point in `\_ingest\_remote\_segments`, nothing has been written. ****This sub-claim is false for partial-row leakage.****

****Scenario B: transient `add\_play\_segment` failure.**** `add\_play\_segment` catches `Exception` and returns `None` (database.py:469) ? it never raises. So this cannot propagate. ****This sub-claim is false.****

****Scenario C: mid-loop raise after some rows written.**** The only code after `add\_play\_segment` succeeds is `n += 1` and loop continuation. `csv.DictReader` iteration could theoretically raise (e.g., `\_csv.Error` on a malformed line, or a decode error), and the row-parse is guarded but `csv.DictReader.\_\_next\_\_` itself runs outside the try. A `UnicodeDecodeError` or `csv.Error` mid-stream (after some rows committed) WOULD propagate uncaught and leave partial rows. This is a narrow but real path.

So the finding's core mechanism ? ****an uncaught exception from `\_ingest\_remote\_segments` AFTER some rows are committed leaves orphaned partial rows**** ? is technically possible (Scenario C), but the two concrete examples the finding leads with (missing intervals.csv, transient add_play_segment) are both wrong because (A) fetch happens before any write, and (B) add_play_segment swallows.

Let me verify the propagation path and what the job worker does. Let me check the worker.

19. user (2026-06-22T15:05:57.508Z)

```
---
backend/api/job_worker.py
```

20. user (2026-06-22T15:06:01.277Z)

```
5:that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
9:every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
46: is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
48: The wiring is additive: the production segmenter path never raises this today (zero
50: (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
72:def _run_claimed_job(job: Dict[str, Any]) -> None:
78:     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means the
86:         result = _main._execute_processing(
92:     except _main.JobWaiting as w:
96:     except Exception as e:
98:         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
128:     except Exception as e: # never wedge a completed job on bookkeeping
148:         _main._run_claimed_job(job)
151:     # on the next submit/drain. Best-effort ? never raises into the worker.
163:     except Exception as e: # pragma: no cover - defensive; never wedge the worker
```

21. user (2026-06-22T15:06:04.341Z)

```
72     def _run_claimed_job(job: Dict[str, Any]) -> None:
73         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
74         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
75
76         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
77         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
79         job self-scheduled and will be re-claimed when `next_poll_at` is due.
80         """
81         import backend.main as _main
82         db_instance = _main.db_instance
83         job_id = job["id"]
84         req = _main._job_to_request(job)
85         try:
86             result = _main._execute_processing(
87                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88             if result.get("video_id"):
89                 db_instance.set_job_video_id(job_id, result["video_id"])
90                 db_instance.mark_job_done(job_id, result)
91                 _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
92         except _main.JobWaiting as w:
93             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
94             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
95             db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
96         except Exception as e:
97             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
98             db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
99
100
101     def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
102         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
103         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
104         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
105         persist it.
106
107         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
108         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
```

```

**0** until a
109         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
110     + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
111         With the placeholder pricebook the cost is 0 regardless.

```

22. assistant (2026-06-22T15:06:16.726Z)

Confirmed. The job worker's `except Exception` at line 96-98 just calls `mark\_job\_failed` ? it does NOT prune segments. And critically, the `\_ingest\_remote\_segments` call at main.py:520 is OUTSIDE any try/except in `\_maybe\_dispatch\_remote` (the only try blocks in that function wrap `resolve\_input\_ref` and `compute.run\_infer`). And in `\_execute\_processing`, the call to `\_maybe\_dispatch\_remote` at line 606 is inside the big `try/finally` (line 564), but the `finally` only emits the report ? it does NOT prune. So an exception from `\_ingest\_remote\_segments` propagates uncaught all the way out.

Now the key question for severity: is there a realistic way for `\_ingest\_remote\_segments` to raise AFTER writing at least one row? Let me confirm the in-process path establishes the contract that this violates, and check whether the watermark protects against the leak on the NEXT re-ingest. Let me also check `query\_play\_segments` filtering used at line 522.

The critical insight on the **destroy-after-confirm violation**: The finding says on a re-ingest the video would have BOTH prior good rows AND orphaned partial rows. Let me think about the watermark mechanics:

- `play\_wm` = watermark captured BEFORE the run (max id of prior rows).
- On success, `\_finalize\_reingest` drops rows with `id <= play\_wm` (the old ones), keeping new ones.
- If `\_ingest\_remote\_segments` writes some rows then raises before `\_finalize\_reingest`, the OLD rows (id <= play_wm) are still present AND the partial NEW rows (id > play_wm) are present. Neither prune ran. So yes ? partial new rows leak, coexisting with old rows.

This is exactly what the in-process path's inner try/except (line 612-619) protects against by calling `prune\_segments(play\_after=play\_wm)` on raise. The cloud path has no such guard around ingest. The finding's core defect mechanism holds.

Let me check the tests to see if this scenario is covered, and verify there's no outer guard I'm missing.

23. user (2026-06-22T15:06:18.456Z)

```

3:Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest the
rally
4:intervals back into the API DB, so /api/query + /api/compile (local stitch) are unchanged ?
all
5:GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json + intervals.csv +
the
39:     """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv + the
done-marker
40:     into the local bucket exactly where the real worker would, so the bucket-poll + ingest
resolve."""
58:         with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
73:     identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
84:     # A real gs:// input would be fetched to hash+validate; return a local fake so that path
is offline.
85:     fake_local = tmp_path / "fetched.mp4"
105:def test_cloud_dispatch_ingests_segments(cloud_env):
193:     to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""

```

24. assistant (2026-06-22T15:06:22.326Z)

The tests cover the happy path (and dispatch-error / returncode failures, which ARE guarded). There is NO test for an ingest-mid-stream failure leaving partial rows. Let me confirm the test file doesn't test the partial-leak case and check `report.json`-only-landed scenario realism. Let me look at the test's fake worker write order to assess the finding's "only report.json landed" claim.

25. user (2026-06-22T15:06:23.276Z)

```
39         """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
40         into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
41
42         def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
43             self.calls: list[list[str]] = []
44             self.video_id, self.returncode = video_id, returncode
45             self.rows = rows if rows is not None else [(2.74, 6.91, 5, "winner"),
46                                                         (8.74, 11.38, 3, "unforced_error")]
47
48         def run_job(self, job_name, argv, *, region, project=None):
49             self.calls.append(list(argv))
50             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
51             done_uri = argv[argv.index("--done-uri") + 1]
52             if self.returncode == 0:
53                 outputs = os.path.join(out_uri, "outputs", self.video_id)
54                 os.makedirs(outputs, exist_ok=True)
55                 with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
56                     json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
57                               "status": "success"}, f)
58                 with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
59                     w = csv.writer(f)
60                     w.writerow(["start", "end", "shot_count", "ending_reason"])
61                     for s, e, sc, er in self.rows:
62                         w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
63                 os.makedirs(os.path.dirname(done_uri), exist_ok=True)
64                 with open(done_uri, "w", encoding="utf-8") as f:
65                     json.dump({"video_id": self.video_id, "returncode": self.returncode,
66                               "segment_count": len(self.rows), "status": "success"}, f)
67             return "exec-1"
68
69
70         @pytest.fixture
71         def cloud_env(tmp_path, monkeypatch):
72             """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
73             identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
```

26. assistant (2026-06-22T15:06:37.699Z)

This confirms the worker writes report.json and intervals.csv only AFTER the job returns returncode 0, and the done-marker is written after both. So in the real flow, the bucket-poll waits for the done-marker. By the time `\_ingest\_remote\_segments` runs, the done-marker exists, which strongly implies intervals.csv was written. The finding's "only report.json landed (push interrupted)" scenario is somewhat undercut by this ordering: the done-marker gates completion, and intervals.csv is written before the done-marker. However, this is a FAKE worker ? the real Cloud Run worker's write/upload ordering and atomicity (separate object uploads to GCS) could still leave intervals.csv missing while the done-marker is present, especially across distinct GCS object PUTs. That's a genuine real-world risk but it's a "missing intervals.csv" case ? fetch raises BEFORE any row write ? no partial leak.

Let me now assess the overall finding precisely. The key question: ****Can `\_ingest\_remote\_segments` raise AFTER committing at least one row?****

The only post-first-row code that can raise uncaught is the ``csv.DictReader`` iteration itself (line 453). A ``_csv.Error`` (e.g., malformed CSV like a NULL byte or unterminated quote mid-stream) or a ``UnicodeDecodeError`` during iteration would raise after earlier rows were committed. This is plausible if the cloud worker produces a truncated/corrupt `intervals.csv` (e.g., upload interrupted mid-file ? valid first rows, then a torn final line). That's a realistic partial-upload outcome and DOES produce the partial-row leak the finding describes.

So the ****mechanism is real****, but the finding's two headline examples are inaccurate:

- "intervals.csv missing ? fetch raises mid-stream" ? fetch is before the loop; no partial rows (the finding even calls this "mid-stream-impossible" parenthetically, contradicting itself).
- "transient `add_play_segment`" ? `add_play_segment` swallows exceptions, never raises.

The genuinely correct trigger (truncated/corrupt CSV mid-iteration) is real but narrower than the finding states. Let me verify the propagation once more and confirm no broader guard. The ``_execute_processing`` try/finally (line 564/646) ? finally only emits report, doesn't catch. The job worker catches and marks failed without pruning. Confirmed: an uncaught ingest exception leaves partial rows + prior rows, violating destroy-after-confirm.

Let me double check there's nothing in ``mark_job_failed`` or a downstream re-ingest that would clean up the orphaned rows.

27. user (2026-06-22T15:06:38.627Z)

```
681: def mark_job_failed(self, job_id: str, error: str) -> bool:
```

28. user (2026-06-22T15:06:41.829Z)

```
681 def mark_job_failed(self, job_id: str, error: str) -> bool:
682     """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
683     return self._execute_write(
684         "UPDATE jobs SET status='failed', error=?, "
685         "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
686         f"Failed to mark job {job_id} failed",
687     )
688
689 def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
690     """Fetch one job; `result`/`player_pool` JSON deserialized."""
691     with self._connect() as conn:
692         row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
```

29. assistant (2026-06-22T15:07:02.568Z)

Confirmed ? ``mark_job_failed`` only updates the jobs table; it does not touch segments. So orphaned partial rows persist.

I now have a complete and adversarially-tested picture. Let me synthesize the verdict.

Verdict

****The finding's core defect is real****, but its supporting examples are partly inaccurate, which affects severity calibration.

****What holds (verified):****

- ``_ingest_remote_segments`` (main.py:520) is genuinely called with NO surrounding try/except. The two ``try`` blocks in ``_maybe_dispatch_remote`` only wrap ``resolve_input_ref`` (495-498) and ``compute.run_infer`` (510-514).
- Rows ARE written one-by-one with per-row commits (``add_play_segment``, database.py:467).
- An uncaught exception propagates through ``_execute_processing`` (the line-606 call sits in the big ``try``, but the ``finally`` at 646 only emits a report ? no prune) ? ``job_worker._run_claimed_job`` ``except Exception`` (line 96-98) which calls ``mark_job_failed`` only, NOT prune. Verified ``mark_job_failed`` touches only the jobs table.
- The in-process path (612-619) DOES guard with ``prune_segments(play_after=play_wm)`` on raise ? so the cloud path is inconsistent and breaks the destroy-after-confirm contract that the rest of

the file carefully upholds.

- Result of a mid-loop raise: prior good rows (id ≤ play_wm) AND partial new rows (id > play_wm) both survive ? exactly the leak described.
- No test covers the ingest-failure-mid-stream case.
- The proposed fix (wrap ingest, prune `play\_after`/`cand\_after`, return `\_failed(...)`) is correct and matches the existing in-process pattern.

****What does NOT hold (the finding's two headline triggers are wrong):****

- "intervals.csv missing ? storage.fetch raises" ? `storage.fetch` (line 450) runs BEFORE the row-writing loop, so this leaves ZERO partial rows, not a partial leak. The finding even writes "mid-stream-impossible" parenthetically, contradicting its own claim.
- "transient add_play_segment" ? `add\_play\_segment` catches `Exception` and returns `None` (database.py:469-471); it never raises, so it cannot propagate.

****The one genuinely correct trigger**** (which the finding does not name): a truncated/corrupt `intervals.csv` causing `csv.DictReader` iteration (line 453) to raise `\_csv.Error`/`UnicodeDecodeError` AFTER earlier rows were committed. A partial GCS upload that lands the done-marker but a torn intervals.csv is a plausible (if narrow) real-world cause.

****Severity:**** Major is slightly high. The defect is real and the fix is warranted (it's a correctness/consistency gap on a data-integrity invariant), but: the cloud path is brand-new and DEFAULT-OFF (`get\_compute\_backend` returns `None` for non-cloud_run targets); the trigger window is narrow (corrupt-mid-file CSV, since the most-cited triggers don't actually leak); and the blast radius is "orphaned rows the next successful re-ingest's `\_finalize\_reingest` `play\_upto` prune would NOT clean (it drops id ≤ new watermark, and the orphans are ≤ the old watermark, so they'd actually be swept on the next SUCCESSFUL run's `play\_upto=new\_wm` ? they're ? that)." That last point softens the persistence: a subsequent successful re-ingest drops everything ? its watermark, which includes the orphans. The leak persists only until the next successful run. I'll rate it ****minor**** ? real, worth fixing for invariant-consistency, but low-likelihood trigger, default-OFF surface, and self-healing on the next success.

30. user (2026-06-22T15:07:11.765Z)

Structured output provided successfully

31. assistant (2026-06-22T15:07:19.891Z)

Verified. The finding holds as a real defect but is over-stated: I downgraded it from major to minor.

Summary:

- ****Real:**** `\_ingest\_remote\_segments` at `C:/Users/avidu/Projects/bhi-cloudrun/backend/main.py:520` has no try/except. A mid-loop raise leaves partial new rows alongside prior good rows, violating the destroy-after-confirm contract that the in-process path explicitly upholds at main.py:612-619. Confirmed the exception propagates uncaught through `\_execute\_processing` (finally at 646 only emits a report) to `job\_worker.\_run\_claimed\_job` (job_worker.py:96-98), which calls `mark\_job\_failed` only ? verified that does not prune segments (database.py:681). The proposed fix is correct.
- ****Two of the finding's three triggers are wrong:**** the missing-intervals.csv case (`storage.fetch` at line 450 runs before any row write ? no partial rows, and the finding itself says "mid-stream-impossible"), and the transient `add\_play\_segment` case (it swallows exceptions and returns `None`, database.py:469-471). The only genuinely-leaking trigger is a corrupt/truncated CSV making `csv.DictReader` iteration raise after some rows committed ? which the finding never names.
- ****Why minor, not major:**** the cloud path is brand-new and default-OFF, the real trigger window is narrow, and the orphaned rows (id ≤ old watermark) get swept by the next successful re-ingest's `\_finalize\_reingest` `play\_upto` prune ? so the leak is bounded and self-healing on the next success, not permanent.